Laboratorium 5

Elementy multimedialne i złudzenia optyczne z wykorzystaniem HTML i CSS

Pliki projektu: lab05/multimedia.htm (zadania 1 i 2), lab05/blocks.html (zadanie 3)

Autor: Artur Gubanovich

Data: 8 listopada 2026 r.

1. Wprowadzenie

Celem ćwiczenia było przerobienie dwóch dostarczonych dokumentów HTML z archiwum lab05.zip tak, aby zapewniały funkcje opisane w zadaniach: sterowanie dźwiękiem i animacją obrazów (zadanie 1), zewnętrzny pasek przewijania filmu (zadanie 2) oraz przyciski włączające i wyłączające elementy złudzenia optycznego (zadanie 3).

Środowisko i weryfikacja: Google Chrome 128 (macOS). Poprawność działania sprawdzono automatycznie skryptem Playwright: odczytano długości nagrań, obliczone style CSS (`transition-duration`, `transform`), wartość `currentTime` filmu po kliknięciu belki oraz atrybuty `aria-pressed` przycisków. Wyniki pomiarów podano w punkcie 5.

Uruchomienie: wystarczy otworzyć pliki `multimedia.htm` lub `blocks.html` w przeglądarce (bez serwera). Ścieżki do mediów są względne, a strony nie wymagają połączenia z Internetem.

1.1. Zmiany ogólne względem plików z materiałów

- Usunięto bibliotekę jQuery ładowaną z adresu `http://code.jquery.com`. Kod przepisano na czysty JavaScript (`addEventListener`, `classList`), więc strony działają offline i nie są blokowane jako treść mieszana (mixed content) na stronach HTTPS.
- Zamieniono deklarację XHTML 1.0 Strict na `<!DOCTYPE html>` z `<meta charset="UTF-8">` i atrybutem `lang="pl"`.
- Dodano obsługę klawiatury i atrybuty dostępności (role, `tabindex`, `aria-*`), ponieważ wszystkie elementy klikalne powinny być dostępne także bez myszy.

2. Zadanie 1 – dźwięki zwierząt i animacja skali

2.1. Wymagania

- pliki dźwiękowe umieszczone jako elementy `<audio>` bez kontrolek (niewidoczne na stronie),
- obrazy zwierząt o jednakowej szerokości 100 px z zachowaniem proporcji,
- kliknięcie obrazu odtwarza plik audio o tej samej nazwie,
- obraz powiększa się płynnie do skali 2x w czasie równym długości nagrania,
- po zdarzeniu zakończenia odtwarzania (`ended`) obraz wraca do rozmiaru pierwotnego w 0,5 s.

2.2. Realizacja

HTML. Dla każdego zwierzęcia dodano element `<audio>` o identyfikatorze `sound` + nazwa obrazka (np. `soundHorse` dla `<img id="Horse">`). Brak atrybutu `controls` powoduje, że przeglądarka nie rysuje

odtwarzacza. Dla konia podano dwa źródła (`horse.ogg` i `horse.mp3`), aby przeglądarka wybrała obsługiwany format.

```
<audio id="soundHorse" preload="auto">
    <source src="audio/horse.ogg" type="audio/ogg">
    <source src="audio/horse.mp3" type="audio/mpeg">
</audio>
<img id="Horse" class="animalButton" src="images/horse.svg"
     alt="Koń" role="button" tabindex="0">
```

Listing 1. Para audio + obraz – nazwy są zgodne, co pozwala powiązać je w skrypcie.

CSS. Obrazy mają `width: 100px; height: auto`, co zachowuje proporcje plików SVG. Domyślny czas tranzycji właściwości `transform` wynosi 0,5 s – jest to czas powrotu. Klasa `.playing` ustawia `transform: scale(2)`. Środek skalowania (`transform-origin: center`) i odstępy w siatce dobrano tak, aby powiększony obraz nie nachodził na sąsiednie.

```
.animalButton {
    width: 100px;  height: auto;
    transform: scale(1);
    transform-origin: center center;
    transition-property: transform;
    transition-duration: .5s;          /* powrót do rozmiaru pierwotnego */
}
.animalButton.playing {
    transform: scale(2);
    transition-timing-function: linear;
}
```

Listing 2. Style obrazu i stanu odtwarzania.

JavaScript. Po kliknięciu skrypt odszukuje element audio (`"sound" + img.id`), ustawia czas tranzycji równy `audio.duration`, dodaje klasę `.playing` i wywołuje `play()`. Długość nagrania jest znana dopiero po wczytaniu metadanych, dlatego w razie potrzeby skrypt czeka na zdarzenie `loadedmetadata`. Zdarzenie `ended` przywraca czas tranzycji 0,5 s i usuwa klasę – obraz wraca do skali 1. Ponowne kliknięcie w trakcie odtwarzania zaczyna dźwięk i animację od początku.

```
function animalClicked(img) {
    const audio = document.getElementById("sound" + img.id);
    const start = () => {
        img.style.transitionDuration = audio.duration + "s"; // = długość ścieżki
        img.classList.add("playing");
        audio.currentTime = 0;
        audio.play();
    };
    if (isNaN(audio.duration)) {
        audio.addEventListener("loadedmetadata", start, { once: true });
        audio.load();
    } else start();
}

audio.addEventListener("ended", () => {               // zdarzenie zakończenia
    img.style.transitionDuration = RETURN_TIME + "s";  // 0.5 s
    img.classList.remove("playing");
});
```

Listing 3. Obsługa kliknięcia i zdarzenia ended (fragment).

2.3. Wynik



Rys. 1. *Zadanie 1 – koń w trakcie powiększania (ok. 1,4 s od kliknięcia, skala ≈ 1,9). Pozostałe obrazy mają szerokość 100 px.*



Rys. 2. *Kolejne klatki animacji koguta. Czas i skala odczytane z `getComputedStyle` w trakcie odtwarzania nagrania rooster.mp3 (2,06 s).*

Skala rośnie liniowo od 1 do 2 dokładnie przez czas trwania nagrania. Po zdarzeniu `ended` (ok. 2,06 s) obraz wraca do skali 1 w ciągu około 0,5 s.

3. Zadanie 2 – zewnętrzny pasek przewijania filmu

3.1. Wymagania

Film umieszczony elementem `<video>`, a pod nim belka (np. o szerokości filmu). Kliknięcie belki ustawia pozycję odtwarzania proporcjonalną do miejsca kliknięcia w osi poziomej. Wizualizacja bieżącej pozycji nie jest wymagana.

3.2. Realizacja

Element `<video>` ma dwa źródła: `video.mp4` (H.264, obsługiwane przez wszystkie przeglądarki) oraz zapasowo `video.mkv`. Belka `#videoProgressBar` ma szerokość 100% kontenera odtwarzacza, czyli dokładnie szerokość filmu. Pozycję kliknięcia liczy się względem lewej krawędzi belki z `getBoundingClientRect()`. Jest to odporniejsze niż `pageX - offsetLeft` z pliku z materiałów, bo uwzględnia przewinięcie strony i zagnieżdżone kontenery.

```
bar.addEventListener("click", e => {
    const rect = bar.getBoundingClientRect();
    const x = e.clientX - rect.left;       // pozycja kliknięcia w belce [px]
    seekToRatio(x / rect.width);           // 0 … 1
```

```
});

function seekToRatio(ratio) {
    ratio = Math.min(Math.max(ratio, 0), 1);
    video.currentTime = ratio * video.duration;
}
```

Listing 4. Przeliczenie miejsca kliknięcia na czas filmu.

Elementy dodatkowe (nieobowiązkowe):

- wypełnienie belki pokazujące bieżącą pozycję, aktualizowane zdarzeniem `timeupdate`, oraz licznik czasu `0:18 / 0:30`,
- pionowy znacznik pod kursorem i podpowiedź (`title`) z czasem, do którego nastąpi przeskok,
- obsługa klawiatury: belka ma `role="slider"` i `tabindex="0"`; strzałki przewijają o 5 s, Home/End na początek/koniec, a `aria-valuenow` i `aria-valuetext` informują czytnik ekranu o pozycji.

3.3. Wynik



Rys. 3. *Zadanie 2 – stan po kliknięciu w 60% szerokości belki: film przeskoczył do 0:18 z 0:30. Pionowa kreska to znacznik pod kursorem.*

4. Zadanie 3 – przyciski sterujące złudzeniem optycznym

4.1. Opis złudzenia

Strona `blocks.html` przedstawia tzw. złudzenie kroków (*footsteps illusion*, opisane przez Stuarta Anstisa). Dwa bloki, biały i czarny, poruszają się ze stałą i identyczną prędkością po tle z pionowych pasów. Obserwator widzi jednak, że bloki poruszają się na zmianę – „krok po kroku", jak stopy idącego człowieka.

Przyczyną jest zmiana kontrastu krawędzi bloku z tłem. Gdy pionowe krawędzie białego bloku leżą na białych pasach, kontrast jest niski i ruch wydaje się wolniejszy. Gdy leżą na czarnych pasach, kontrast jest wysoki i ruch wydaje się szybszy. Dla czarnego bloku zależność jest odwrotna, więc bloki „przyspieszają" naprzemiennie. Wyłączenie tła albo dodanie ramki o stałym kontraście (czerwonej) usuwa złudzenie i pokazuje, że ruch jest jednostajny.

4.2. Realizacja

Dodano grupę trzech elementów `<button>` z atrybutem `data-class`, który wskazuje klasę przełączaną na elemencie `#track`:

Tabela 1. *Przyciski i przełączane klasy.*

Przycisk	Klasa na #track	Efekt
Background	`proof`	Usuwa obraz tła w paski (`background-image: none`). Przycisk wciśnięty = tło widoczne.
White box border	`border-light`	Czerwona ramka 2 px wokół białego bloku.
Black box border	`border-dark`	Czerwona ramka 2 px wokół czarnego bloku.

Bloki mają `box-sizing: border-box`, dzięki czemu włączenie ramki nie zmienia ich rozmiaru ani toru ruchu. Stan przycisku jest przechowywany w atrybucie `aria-pressed`, który jednocześnie steruje wyglądem w CSS. Czytnik ekranu ogłasza więc przycisk jako przełącznik „wciśnięty / niewciśnięty".

Wygląd przycisków. Zaokrąglenie uzyskano przez `border-radius: 8px`. Efekt trójwymiarowy opiera się na założeniu, że światło pada z góry:

- **opcja wyłączona – przycisk wypukły:** `linear-gradient` od jasnego (u góry) do ciemnego (u dołu), zewnętrzny cień pod przyciskiem, jasny `text-shadow` poniżej liter (tekst „wytłoczony"),
- **opcja włączona – przycisk wklęsły:** gradient odwrócony (ciemniej u góry), wewnętrzny cień `inset`, przesunięcie tekstu o 1 px w dół i `text-shadow` nad literami (tekst „wciśnięty").

```
.toggle {                                   /* wyłączony = wypukły */
    border-radius: 8px;
    border: 1px solid #8e9a80;
    background-color: #c6d1b8;
    background: linear-gradient(to bottom,
                #f3f7ee 0%, #d6dfca 45%, #b9c6a8 55%, #a4b291 100%);
    box-shadow: inset 0 1px 0 rgba(255,255,255,.8), 0 2px 3px rgba(0,0,0,.35);
    text-shadow: 0 1px 0 rgba(255,255,255,.8);
}
.toggle[aria-pressed="true"] {              /* włączony = wklęsły */
    background: linear-gradient(to bottom,
                #93a17f 0%, #aab896 45%, #bfcbaf 55%, #d9e2cf 100%);
    box-shadow: inset 0 2px 5px rgba(0,0,0,.45), 0 1px 0 rgba(255,255,255,.4);
    text-shadow: 0 -1px 0 rgba(255,255,255,.35);
```

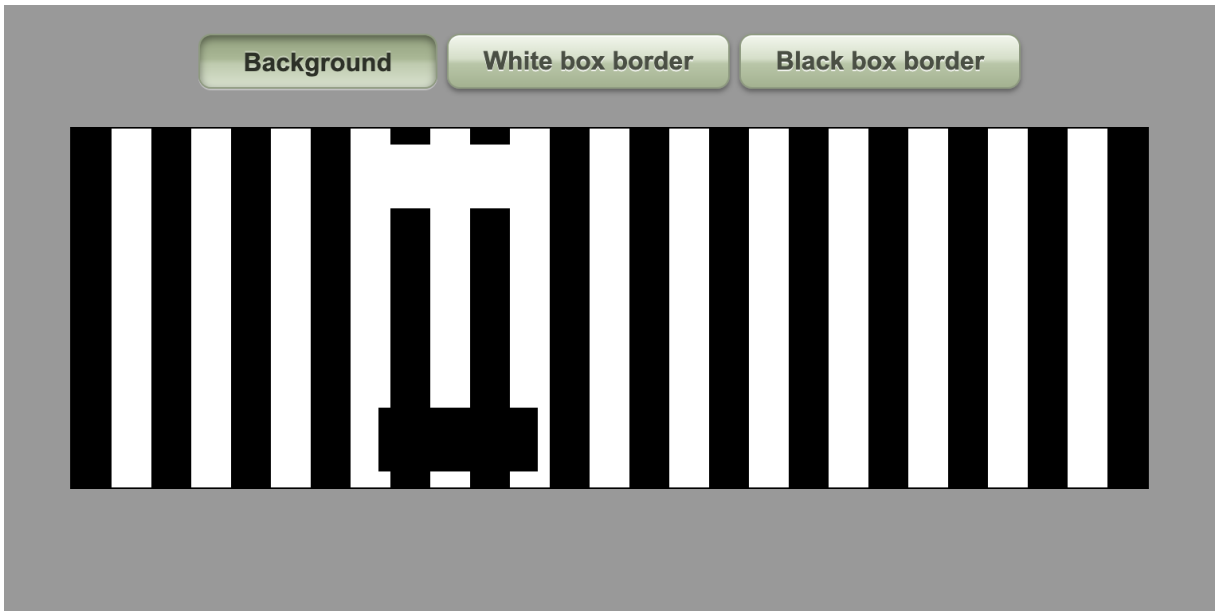
```
}
```

Listing 5. Style przycisków wypukłych i wklęsłych.

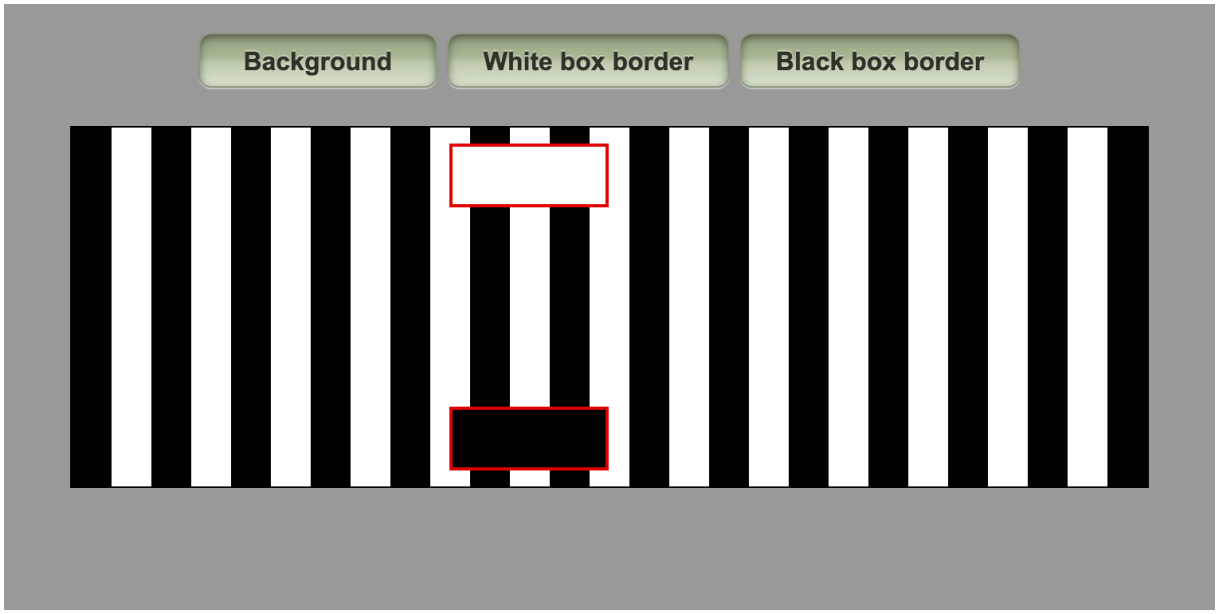
```
document.querySelectorAll(".toggle").forEach(btn => {
    sync(btn);
    btn.addEventListener("click", () => {
        track.classList.toggle(btn.dataset.class);
        sync(btn);                      // aktualizacja aria-pressed
    });
});
```

Listing 6. Przełączanie klas.

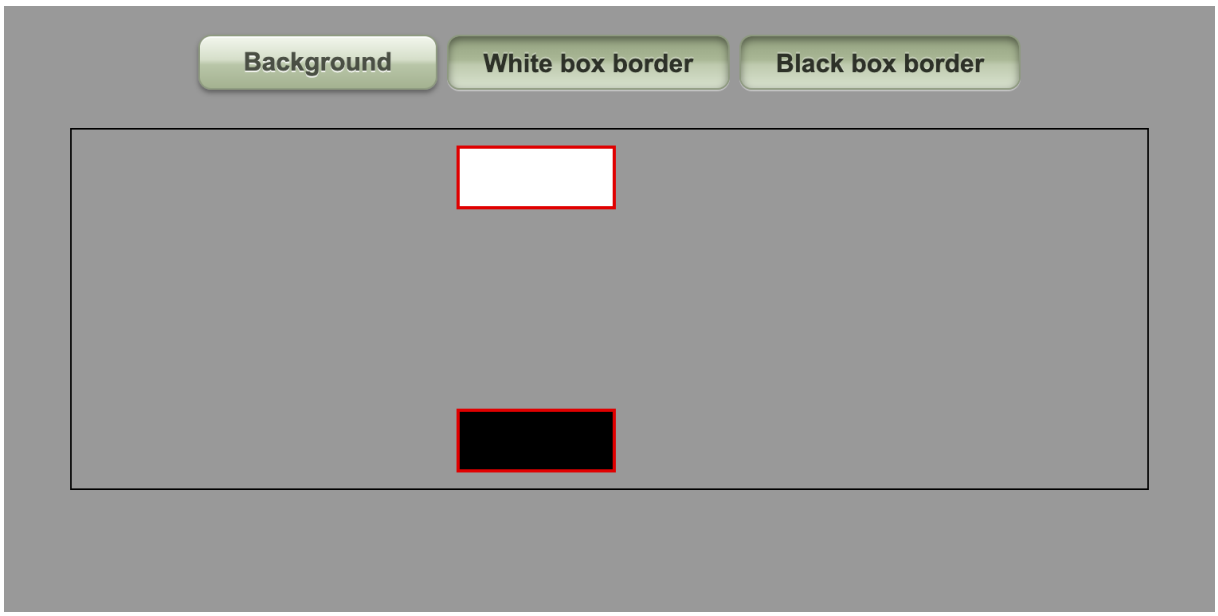
4.3. Wynik



Rys. 4. *Stan początkowy: tło włączone (przycisk wklęsły), ramki wyłączone (przyciski wypukłe). Bloki wydają się poruszać na zmianę.*



Rys. 5. *Włączone ramki obu bloków – czerwona krawędź ma stały kontrast z tłem, więc złudzenie zanika.*



Rys. 6. *Wyłączone tło (przycisk „Background" wypukły) – bloki poruszają się wyraźnie równo.*

5. Weryfikacja

Tabela 2. *Zestawienie wymagań i wyników testów.*

Wymaganie	Sposób sprawdzenia	Wynik
Audio bez kontrolek	Brak atrybutu `controls`, zrzut strony	Spełnione – brak odtwarzaczy na stronie
Szerokość obrazów 100 px, proporcje zachowane	`width:100px; height:auto`	Spełnione
Odtworzenie właściwego dźwięku	`currentSrc` dla każdego audio	elephant.mp3, horse.ogg, rooster.mp3, frog.wav
Czas powiększania = długość nagrania	`transitionDuration` vs `audio.duration` (kogut)	2,06367 s = 2,063673 s
Skala 2x	`getComputedStyle().transform`	1,00 → 1,99 w chwili `ended`
Powrót w 0,5 s po `ended`	`transitionDuration` po zdarzeniu	0,5 s; skala 1,00 po ok. 0,6 s
Przewijanie filmu kliknięciem belki	Kliknięcie w 60% szerokości	`currentTime` = 18,598 s (oczekiwane 0,6 × 30,997 = 18,598 s)
Klawisz Home na belce	`currentTime` po naciśnięciu	0 s
3 przyciski przełączające	klasy `#track` i `aria-pressed` po kliknięciach	proof / border-light / border-dark przełączane niezależnie
Wypukłość / wklęsłość	Zrzuty ekranu obu stanów	Spełnione (rys. 4–6)

Ograniczenie: odgłosy zwierząt są krótkie (0,72–2,06 s), więc animacja żaby trwa niecałą sekundę. Jest to zgodne z wymaganiem, bo czas tranzycji ma być równy długości ścieżki.

6. Wnioski

- API elementów `<audio>` i `<video>` (`play()`, `duration`, `currentTime`, zdarzenia `loadedmetadata`, `timeupdate`, `ended`) pozwala zbudować własny interfejs odtwarzania bez zewnętrznych bibliotek.
- Połączenie przejść CSS z wartościami ustawianymi w JavaScripcie (czas tranzycji = długość dźwięku) daje spójną informację zwrotną: rozmiar obrazu pokazuje, jak długo jeszcze trwa nagranie.
- Gradienty i cienie są skutecznym, skeuomorficznym sygnałem stanu przełącznika (affordance). Stan wizualny powinien jednak wynikać z atrybutu semantycznego (`aria-pressed`), aby był dostępny także dla czytników ekranu.
- Złudzenie kroków pokazuje, że postrzegana prędkość zależy od kontrastu krawędzi z tłem. W interfejsach animowane elementy na wzorzystym tle mogą wydawać się szarpane, choć poruszają się płynnie.

Źródła

- W3Schools: HTML Audio/Video DOM Reference, CSS Transitions, CSS 2D Transforms, CSS Gradients.
- Anstis S. (2001). Footsteps and inchworms: Illusions show that contrast affects apparent speed. *Perception*, 30(7), 785–794.
- Ikony: Elephant i Frog – Tatiana Belkina, Horse – Samy Menai (the Noun Project).